

UNIDAD DIDÁCTICA 6

PROGRAMACIÓN DE COMPONENTES DE ACCESO A DATOS

MÓDULO PROFESIONAL:
ACCESO A DATOS



CESUR
Tu Centro Oficial de FP

Índice

RESUMEN INTRODUCTORIO	2
INTRODUCCIÓN	2
CASO INTRODUCTORIO	3
1. PROGRAMACIÓN ORIENTADA A COMPONENTES: PROPIEDADES Y CARACTERÍSTICAS BÁSICAS.....	4
1.1 Eventos.....	10
1.2 Persistencia	15
1.3 Introspección y reflexión	18
2. PLATAFORMAS Y MODELOS DE COMPONENTES	29
2.1 Herramientas para el desarrollo de componentes.....	31
2.2 Empaquetado de componentes.....	36
2.3 Utilización de componentes	39
2.4 Prueba y documentación de los componentes desarrollados	39
RESUMEN FINAL	44

RESUMEN INTRODUCTORIO

En esta unidad trabajaremos por primera vez el concepto de componente. Los componentes no son exclusivos de la persistencia, puesto que se pueden aplicar a cualquier parte de una aplicación. Después de ver el concepto de componente y analizar las características, introduciremos la relación de los componentes con los conceptos de persistencia. Se trata de componentes específicos encargados del almacenamiento y recuperación de los datos de las aplicaciones.

Revisaremos las plataformas y los estándares que tenemos en Java para poder implementar estos componentes y que puedan ser usados tanto por aplicaciones propias como por terceros.

Se verán las herramientas más utilizadas en el desarrollo de componentes y cómo trabajar con estos, incluyendo el empaquetado de componentes. Con todo este conocimiento, finalizaremos con un apartado de pruebas y documentación de dichos componentes, y de esta forma tenerlos preparados para su uso.

INTRODUCCIÓN

A medida que nos tengamos que enfrentar a aplicaciones cada vez más complejas, nos daremos cuenta de que el número de clases empezará a crecer desmesuradamente y que conseguir cohesionarlas, aplicando criterios de reutilización, eficiencia y calidad, es una tarea realmente difícil.

Para intentar salvar esta dificultad, apareció el concepto de componente de software. La idea es tratar el software como si fuera un producto mecánico. En la industria mecánica, estos sistemas se basan en hacer construcciones parciales, organizadas de tal manera que todas se puedan realizar a la vez y que, una vez acabado su ensamblaje, dé como resultado el producto final. Cada una de estas construcciones se denomina componente.

De forma parecida, si fuera posible organizar todo el conjunto de clases que componen una aplicación en componentes independientes, de forma que fuera posible la creación concurrente de cada pieza y se consiguiera un ensamblaje final fácil y eficiente, nos encontraríamos ante una posible solución al problema de la construcción de aplicaciones complejas.

CASO INTRODUCTORIO

Trabajas como desarrollador autónomo con varios clientes a los que realizas mantenimientos y desarrollos individualizados para sus empresas.

Tienes diversos clientes que provienen del mismo sector, el de hostelería, todos ellos con necesidades muy parecidas, para los cuales has hecho desarrollos destinados a la gestión de sus productos de cocina.

Para mejorar el mantenimiento de los desarrollos, su reutilización y actualizaciones futuras, nos planteamos realizar una revisión del código.

Al finalizar la unidad, conocerás las bases del concepto de componente, así como sabrás planificar y desarrollar un componente.

1. PROGRAMACIÓN ORIENTADA A COMPONENTES: PROPIEDADES Y CARACTERÍSTICAS BÁSICAS

Tu código para los diferentes clientes está ya estructurado con Java y POO. Sin embargo, queremos dar ese paso de modularidad planificando y programando componentes.

Sabes que la principal ventaja de los componentes es que facilitan la reutilización y el mantenimiento del software. En este caso, para el sector hostelero, buscas componentes que sean versátiles y capaces de adaptarse a cualquier tipo de negocio de hostelería. Así garantizarás que tus componentes implementen eventos, verificarás que te permitan hacer introspección y reflexión y, por último, que garanticen la persistencia de los datos de los restaurantes.

La **programación orientada a objetos (POO)**, tal y como se ha estudiado durante el ciclo, tiene muchas ventajas. Aunque, sin embargo, presenta determinadas deficiencias a la hora de desarrollar componentes:

- Las reutilizaciones de objetos son complicadas.
- No incorpora aspectos tales como distribución y empaquetado de componentes.
- Imposibilidad de separar aspectos composicionales y computacionales.
- No define una unidad de composición de aplicaciones software.

Por estas razones, surgió la programación orientada a componentes (POC), considerada como una extensión de la programación orientada a objetos para sistemas abiertos y distribuidos.

Su principal objetivo es permitir el desarrollo de aplicaciones reutilizando componentes ya realizados y probados. De esta forma, se **agiliza el proceso de desarrollo del software**.

Entre las **ventajas** de la POC destacarían:

- Reutilización de componentes.
- Al desarrollar cada componente, se prueba por separado, lo que facilita el proceso de pruebas y el posterior mantenimiento.
- Posibilidad de actualizar y/o agregar componentes a una aplicación en función de la necesidad.
- Un componente puede, una vez construido, mejorarse continuamente.
- El retorno sobre la inversión mejorará notablemente si se aplica correctamente este paradigma.

Aunque es un paradigma en auge, presenta ciertas **desventajas**:

- En principio, el desarrollador de componentes no sabe para quién está dirigido el componente ni como lo utilizará, lo que le supone una dificultad añadida.
- Además, cuando finalmente el usuario del componente lo utilice, se encontrará también con el problema de no poder adaptarlo a sus necesidades, ya que no tiene acceso a su implementación.
- A esto, hay que añadir que aún no existe una solución estándar al problema de la convivencia entre versiones de un mismo componente.
- Por último, se debe citar el hecho de que falta soporte para el desarrollo de componentes.

El **futuro de la POC** se está centrando en:

- Incorporar los conceptos propios de este paradigma a los paradigmas ya existentes.
- Diseñar lenguajes de programación propios, ya que son pocos los lenguajes de programación que incorporan orientación a componentes, entre ellos: Java, Ada95, Modula-3 y Oberon. La mayoría no son completamente orientados a componentes, sino que incorporan conceptos propios de este paradigma.
- Construir herramientas de desarrollo para componentes.



ENLACE DE INTERÉS

En esta web se puede encontrar más información sobre la POC: concepto, características y problemas:





PARA SABER MÁS

Antes de seguir avanzando en la programación orientada a componentes, es recomendable recordar los conceptos básicos de la programación orientada a objetos:



La unidad mínima de composición de este paradigma es el componente. *“Un componente es una unidad de composición de aplicaciones software, que posee un conjunto de interfaces y un conjunto de requisitos, y que ha de poder ser desarrollado, adquirido, incorporado al sistema y compuesto con otros componentes de forma independiente, en tiempo y espacio”* (Szyperski, 1998).

Un componente puede controlarse de manera dinámica y ensamblarse construyendo una aplicación software. Además, posee una funcionalidad que puede ser reutilizada en diferentes aplicaciones y manipulada por una herramienta de programación. Debido a esto, la plataforma o entorno de desarrollo de componentes puede interrogar al componente para conocer sus propiedades y los eventos que puede generar en respuesta a las distintas acciones.

Una propiedad de un componente es un atributo que afecta a su comportamiento o apariencia. Un ejemplo de propiedad podría ser el color del texto, el tamaño o la posición.

JavaBeans es un patrón de diseño en Java que define una clase como un componente reutilizable y encapsulado. Los JavaBeans son una parte importante de la programación orientada a componentes en Java. Con JavaBeans, las propiedades pueden ser:

- **Simples:** aquellas que representan un único valor. Los métodos utilizados son `get...()` y `set...()`. El siguiente ejemplo muestra cómo usar una propiedad simple.

```
//miembro de la clase que se usa para guardar el valor de la propiedad  
private String nombre;
```

```
//métodos set y get de la propiedad denominada Nombre  
public void setNombre(String nuevoNombre) {
```

```
        nombre=nuevoNombre;
    }

    public String getNombre() {
        return nombre;
    }
}
```

- **Indexadas:** aquellas que tienen asociadas más de un valor. En este ejemplo, se muestra cómo definir y usar una propiedad indexada.

```
private int[] numeros = {1,2,3,4};

//métodos set y get de la propiedad denominada Numeros, para el array completo
public void setNumeros (int[] nuevoValor) {
    numeros=nuevoValor;
}

public int[] getNumeros() {
    return numeros;
}
```

- **Ligadas:** aquellas que, al cambiar, se informa a otros componentes, permitiéndoles realizar alguna acción. En este caso, la notificación del cambio se realiza mediante un `PropertyChangeEvent`. Los componentes que quieran ser notificados del cambio en una propiedad deberán registrarse como auditores.

Ejemplo de propiedad ligada:

```
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

public class Persona {
    private String nombre;
    private final PropertyChangeSupport support;

    public Persona() {
        support = new PropertyChangeSupport(this);
    }

    // Métodos set y get de la propiedad nombre
    public void setNombre(String nuevoNombre) {
        String nombreViejo = this.nombre;
        this.nombre = nuevoNombre;

        // Notificar a los escuchadores del cambio en la propiedad
        support.firePropertyChange("nombre", nombreViejo,
nuevoNombre);
    }
}
```



```
public String getNombre() {
    return nombre;
}

// Método para agregar un escuchador de cambios de propiedad
public void addChangeListener(PropertyChangeListener
listener) {
    support.addChangeListener(listener);
}

// Método para remover un escuchador de cambios de propiedad
public void removeChangeListener(PropertyChangeListener
listener) {
    support.removeChangeListener(listener);
}
}
```

Uso de propiedad ligada:

```
public class Main {
    public static void main(String[] args) {
        Persona persona = new Persona();

        // Agregar un escuchador que reacciona cuando cambia el nombre
        persona.addChangeListener(evt -> {
            System.out.println("El nombre ha cambiado de " +
            evt.getOldValue() + " a " + evt.getNewValue());
        });

        // Cambiar el nombre, lo que disparará el evento
        persona.setNombre("Juan");
        persona.setNombre("Carlos");
    }
}
```

- **Restringidas:** concepto similar al de propiedad ligada; la diferencia está en que los componentes a los que se les informa del cambio de la propiedad tienen la opción de vetar el cambio.

Ejemplo de propiedad restringida:

```
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.beans.PropertyVetoException;
import java.beans.VetoableChangeListener;
import java.beans.VetoableChangeSupport;

public class Persona {
    private String nombre;
    private final PropertyChangeSupport support;
```

```
private final VetoableChangeSupport vetoableSupport;

public Persona() {
    support = new PropertyChangeSupport(this);
    vetoableSupport = new VetoableChangeSupport(this);
}

// Métodos set y get de la propiedad nombre
public void setNombre(String nuevoNombre) throws
PropertyVetoException {
    String nombreViejo = this.nombre;

    // Notificar a los escuchadores que pueden vetar el cambio
    vetoableSupport.fireVetoableChange("nombre", nombreViejo,
nuevoNombre);

    // Si no fue vetado, se cambia el nombre
    this.nombre = nuevoNombre;

    // Notificar a los escuchadores del cambio en la propiedad
    support.firePropertyChange("nombre", nombreViejo,
nuevoNombre);
}

public String getNombre() {
    return nombre;
}

// Métodos para agregar y remover escuchadores de cambios de
propiedad
public void addPropertyChangeListener(PropertyChangeListener
listener) {
    support.addPropertyChangeListener(listener);
}

public void removePropertyChangeListener(PropertyChangeListener
listener) {
    support.removePropertyChangeListener(listener);
}

// Métodos para agregar y remover escuchadores que pueden vetar
cambios
public void addVetoableChangeListener(VetoableChangeListener
listener) {
    vetoableSupport.addVetoableChangeListener(listener);
}

public void removeVetoableChangeListener(VetoableChangeListener
listener) {
    vetoableSupport.removeVetoableChangeListener(listener);
}
}
```

Uso de propiedad restringida:

```
import java.beans.PropertyVetoException;

public class Main {
    public static void main(String[] args) {
        Persona persona = new Persona();

        // Agregar un escuchador que puede vetar el cambio de nombre
        persona.addVetoableChangeListener(evt -> {
            String nuevoNombre = (String) evt.getNewValue();
            if (nuevoNombre.equals("Prohibido")) {
                throw new PropertyVetoException("El nombre no puede
ser 'Prohibido'", evt);
            }
        });

        // Agregar un escuchador que reacciona cuando el nombre cambia
        persona.addPropertyChangeListener(evt -> {
            System.out.println("El nombre ha cambiado de " +
evt.getOldValue() + " a " + evt.getNewValue());
        });

        try {
            // Cambiar el nombre correctamente
            persona.setNombre("Juan");

            // Intentar un cambio de nombre que será vetado
            persona.setNombre("Prohibido");
        } catch (PropertyVetoException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

A continuación, se estudiarán los eventos y se verá un ejemplo de clase con propiedades ligadas y cómo gestionarlas.

Entre las principales características de la arquitectura basada en componentes destacan:

- Eventos.
- Persistencia.
- Introspección y reflexión.

1.1 Eventos

Un **evento** es un mecanismo utilizado por los componentes para comunicarse entre sí. Son mensajes que se envían entre componentes notificando que algo ha ocurrido.

La responsabilidad de gestionar los eventos que suceden en un componente (Fuente) la tiene otro componente (Oyente). El componente Fuente del evento invoca a un método del Oyente, pasándole como argumento un objeto que almacena toda la información sobre el evento.

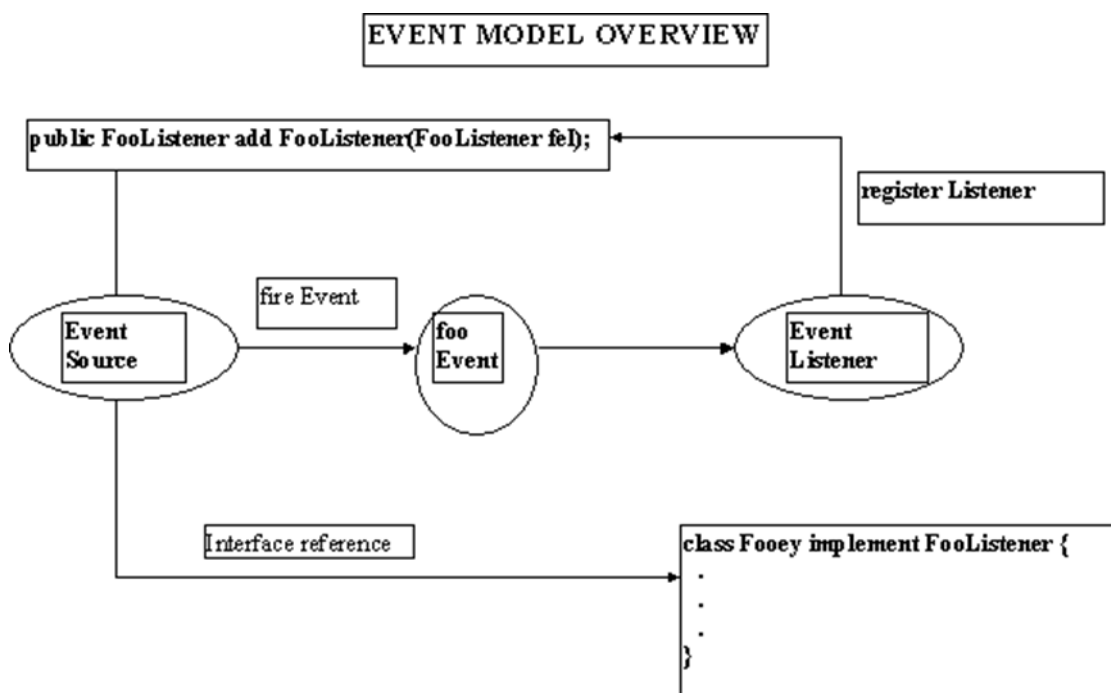
Componente Fuente (Source): origina o lanza los eventos. Los eventos que puede lanzar se encuentran definidos en su API. Además, cuenta con una interfaz para registrar oyentes para cada tipo de evento:

- Para establecer un único oyente: `set<TipoEvento>Listener`
- Para establecer varios oyentes: `add<TipoEvento>Listener`
- Para eliminar un oyente: `remove<TipoEvento>Listener`

Componente Oyente (Listener): gestiona o responde a eventos. Su función es definir los métodos a ser invocados por la fuente en respuesta a cada evento específico que suceda.

- Objeto que implementa la interfaz `<TipoEvento>Listener`

Componentes (Listeners) de Eventos: implementan la interfaz correspondiente al tipo de evento a escuchar: `ActionListener`, `WindowListener`, `MouseListener`, etc.



Eventos en JavaBeans.

Fuente: <https://condor.depaul.edu/elliott/513/projects-archive/DS420Fall98/Paris/~ejones2.html>

Las clases que implementan estas interfaces tienen que implementar TODOS los métodos de la interfaz.

La asociación de acciones a eventos es el proceso por el cual se vincula un evento que ocurre en un componente (Fuente) con una acción que debe ejecutarse como respuesta a dicho evento en otro componente (Oyente). Para establecer esta relación, es necesario registrar un oyente que sea capaz de "escuchar" el evento generado por la fuente. Este oyente implementa una interfaz específica según el tipo de evento y define el comportamiento que se ejecutará cuando ocurra el evento.

Proceso de asociación:

- **Registro del oyente:** la fuente del evento ofrece métodos como `add<Evento>Listener` para asociar uno o más oyentes a un evento particular.
- **Lanzamiento del evento:** cuando la fuente detecta que ha ocurrido el evento, invoca a los oyentes registrados, pasando un objeto que encapsula la información relevante sobre el evento.
- **Respuesta del oyente:** el oyente reacciona a través de un método predefinido, ejecutando la acción asociada al evento.



EJEMPLO PRÁCTICO

Dentro de nuestra aplicación, un punto muy importante para nuestros clientes es conocer en todo momento los movimientos de Stock de los productos.

¿Cómo implementar un Listener para un determinado producto?

En primer lugar, crearemos la clase que después se utilizará en las sincronizaciones:

```
import java.util.*;
public class StockEvent extends EventObject {
    protected int anteStock, nuevoStock;

    public StockEvent (Object fuente , int anterior, int nuevo) {
        super(fuente);
        nuevoStock=nuevo;
        anteStock=anterior;
    }

    public int getNuevoStock () { return nuevoStock;}
}
```

```
    public int getAnteStock () { return anteStock; }  
}
```

A partir de esta clase, podemos usarla en una clase producto:

```
import java.beans.*;  
import java.util.*;  
  
// Clase Producto  
public class Producto {  
    private Vector stockListeners=new Vector();  
    private int stock;  
    public Producto () {  
        stock=20;  
    }  
    public void setStock(int nuevoStock) {  
        int anteStock= stock;  
        stock =nuevoStock ;  
        if(anteStock!=nuevoStock) {  
            StockEvent event=new StockEvent (this, anteStock,  
nuevoStock);  
            notificarCambio(event);  
        }  
    }  
  
    public int getStock () {  
        return stock;  
    }  
    public synchronized void addStockListener(StockListener  
listener) {  
        stockListeners.addElement(listener);  
    }  
    public synchronized void removeStockListener(StockListener  
listener){  
        stockListeners.removeElement(listener);  
    }  
    private void notificarCambio(StockEvent event) {  
        Vector lista;  
        synchronized(this) {  
            lista=(Vector)stockListeners.clone();  
        }  
        for(int i=0; i< lista.size(); i++){  
            StockListener  
listener=(StockListener)lista.elementAt(i);  
            listener.enteradoCambioStock(event);  
        }  
    }  
}
```

Por último, crearíamos el Listener y la interfaz:

```
import java.util.*;  
  
public interface StockListener extends EventListener {
```

```

        public void enteradoCambioStock(EventObject ev) ;
    }

import java.util.*;

// Listener
public class Almacen implements StockListener{
    public Almacen () {}
    public void enteradoCambioStock(EventObject ev) {
        StockEvent event=(StockEvent)ev;
        System.out.println("Almacen: nuevo stock
"+event.getNuevoStock());
        System.out.println("Almacen: stock anterior
"+event.getAnteStock());
    }
}

```

Para enlazar uno con otro objeto, se realizaría de la siguiente forma:

```

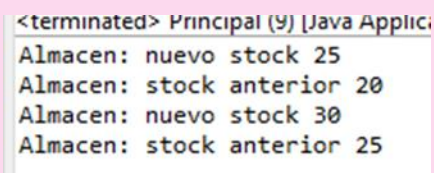
public class Principal {

    public static void main(String[] args) {
        Almacen almacenCentral = new Almacen();
        Producto arroz = new Producto();

        // Asociación del evento
        arroz.addStockListener(almacenCentral);

        // Cambios de stock para probar los eventos
        arroz.setStock(25);
        arroz.setStock(30);
    }
}

```

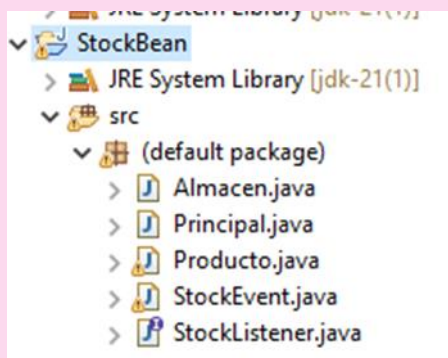


```

<terminated> Principal (9) [Java Applic
Almacen: nuevo stock 25
Almacen: stock anterior 20
Almacen: nuevo stock 30
Almacen: stock anterior 25

```

Ejecución del programa.



Estructura de proyecto de eventos de componentes.

1.2 Persistencia

La **persistencia** es un mecanismo de la programación orientada a componentes, ya que permite guardar y restaurar el estado de los componentes, junto con sus valores personalizados, para su uso posterior o su reutilización en diferentes sesiones de ejecución. Esto nos permite que los componentes puedan mantener su estado a lo largo del tiempo o ser reutilizados en otras aplicaciones, incluso después de haber sido cerrados o transferidos.

Para que un componente sea persistente en Java, debe implementar la interfaz `Serializable`, que permite convertir un objeto en una secuencia de bytes para almacenarlo o transferirlo.



RECUERDA

La serialización consiste en codificar un objeto en un medio de almacenamiento (como puede ser un archivo o un buffer de memoria) con el fin de transmitirlo a través de una conexión en red, como una serie de bytes, o en un formato más legible, como XML, entre otros. La serie de bytes o el formato pueden ser usados para crear un nuevo objeto que es idéntico en todo al original, incluido su estado interno (por tanto, el nuevo objeto es un clon del original).



EJEMPLO PRÁCTICO

Nuestro jefe nos pide crear un mecanismo de Serialización de productos mediante ficheros. ¿Cómo lo implementaríamos?

En primer lugar, crearemos la clase que representará a los productos:

```
import java.io.Serializable;

public class Producto implements Serializable {

    private String nombre;
    private double precio;
    private int cantidad;

    // Constructor sin argumentos (requerido por el estándar
    // JavaBeans)
    public Producto() {}
```



```
// Constructor para inicializar los atributos
public Producto(String nombre, double precio, int cantidad) {
    this.nombre = nombre;
    this.precio = precio;
    this.cantidad = cantidad;
}

// Métodos getter y setter (requerido por el estándar JavaBeans)
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public double getPrecio() {
    return precio;
}

public void setPrecio(double precio) {
    this.precio = precio;
}

public int getCantidad() {
    return cantidad;
}

public void setCantidad(int cantidad) {
    this.cantidad = cantidad;
}

@Override
public String toString() {
    return "Producto{" +
        "nombre='" + nombre + '\'' +
        ", precio=" + precio +
        ", cantidad=" + cantidad +
        '}';
}
}
```

Después, crearemos la clase que se encargará de la persistencia de los productos:

```
import java.io.*;

public class ProductoPersistencia {

    public void serializar(Producto producto, String archivo) {
        try (FileOutputStream fileOut = new
FileOutputStream(archivo);
            ObjectOutputStream out = new
ObjectOutputStream(fileOut)) {
```

```
        out.writeObject(producto);
        System.out.println("El objeto Producto ha sido
serializado y guardado en " + archivo);

    } catch (IOException i) {
        i.printStackTrace();
    }
}

public Producto deserializar(String archivo) {
    Producto producto = null;
    try (FileInputStream fileIn = new FileInputStream(archivo);
        ObjectInputStream in = new ObjectInputStream(fileIn)) {

        producto = (Producto) in.readObject();
        System.out.println("El objeto Producto ha sido
deserializado desde " + archivo);

    } catch (IOException | ClassNotFoundException i) {
        i.printStackTrace();
    }
    return producto;
}
}
```

Por último, crearemos la clase principal que interactuará con todo.

```
public class Principal {
    public static void main(String[] args) {
        // Crear una instancia del JavaBean Producto
        Producto producto = new Producto("Ordenador", 999.99, 10);

        // Nombre del archivo donde se guardará el objeto
        serializado
        String archivo = "producto.ser";

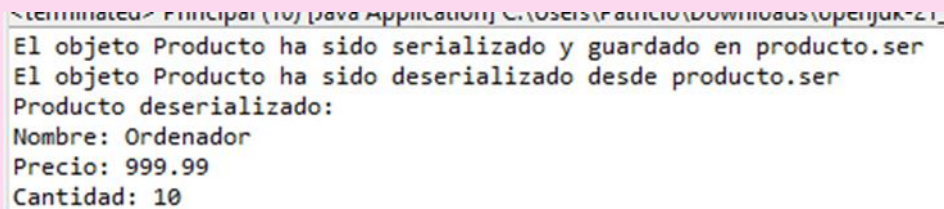
        // Crear una instancia de la clase de serialización y
        deserialización
        ProductoPersistencia persistencia = new
        ProductoPersistencia();

        // Serializar el objeto Producto
        persistencia.serializar(producto, archivo);

        // Deserializar el objeto Producto y mostrar sus detalles
        Producto productoDeserializado =
        persistencia.deserializar(archivo);
        if (productoDeserializado != null) {
            System.out.println("Producto deserializado:");
            System.out.println("Nombre: " +
            productoDeserializado.getNombre());
            System.out.println("Precio: " +
            productoDeserializado.getPrecio());
        }
    }
}
```

```
        System.out.println("Cantidad: " +  
        productoDeserializado.getCantidad());  
    }  
}
```

Si ejecutamos, veremos el resultado:



```
El objeto Producto ha sido serializado y guardado en producto.ser  
El objeto Producto ha sido deserializado desde producto.ser  
Producto deserializado:  
Nombre: Ordenador  
Precio: 999.99  
Cantidad: 10
```

Ejecución de la persistencia.

1.3 Introspección y reflexión

Introspección y reflexión son mecanismos que permiten a un programa obtener información sobre su estructura en tiempo de ejecución y, aunque están relacionados, presentan diferencias importantes.

La **introspección** es un proceso mediante el cual un entorno de desarrollo puede obtener información sobre los métodos, propiedades y eventos de un componente sin ejecutar su código. Este mecanismo es muy usado en entornos de desarrollo como NetBeans o Eclipse, donde los desarrolladores pueden manipular componentes de forma gráfica sin necesidad de conocer su implementación interna.

La introspección se suele realizar a través de la clase `BeanInfo`, que proporciona detalles sobre un `JavaBean`, como sus métodos, propiedades y eventos. Si una clase no proporciona su propia clase `BeanInfo`, el sistema de introspección utiliza mecanismos automáticos para deducir esta información.

Ejemplo de Introspección:

```
public class Persona {  
    private String nombre;  
    private String direccion;  
  
    public String getNombre() { return nombre; }  
    public void setNombre(String nombre) { this.nombre = nombre; }  
  
    public String getDireccion() { return direccion; }  
    public void setDireccion(String direccion) { this.direccion =  
direccion; }  
}
```

```
import java.beans.*;

public class PersonaBeanInfo extends SimpleBeanInfo {
    public PropertyDescriptor[] getPropertyDescriptors() {
        try {
            PropertyDescriptor nombre = new
PropertyDescriptor("nombre", Persona.class);
            PropertyDescriptor direccion = new
PropertyDescriptor("direccion", Persona.class);
            return new PropertyDescriptor[] { nombre, direccion };
        } catch (IntrospectionException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

La clase `PersonaBeanInfo` permite que una herramienta de desarrollo sepa que la clase `Persona` tiene las propiedades `nombre` y `direccion`, incluso sin ejecutar la clase `Persona`.



ENLACE DE INTERÉS

Aquí puedes encontrar la documentación oficial de la clase `BeanInfo`:



Por otra parte, tenemos la reflexión, que es un mecanismo más avanzado, ya que permite a un programa inspeccionar y modificar su propia estructura (clases, métodos, campos) en tiempo de ejecución. A diferencia de la introspección, que está enfocada en la descripción de las propiedades y eventos de un componente, la reflexión permite acceder y modificar clases y objetos de forma dinámica durante la ejecución.

En Java, tenemos la API de reflexión en el paquete `java.lang.reflect`, que permite descubrir clases, métodos y campos, e incluso invocarlos o modificarlos en tiempo de ejecución.

Ejemplo de reflexión:

```
import java.lang.reflect.*;

public class ReflexionEjemplo {
    public static void main(String[] args) {
        try {
            // Obtener la clase Persona mediante reflexión
            Class<?> clase = Class.forName("Persona");

            // Crear una instancia de la clase Persona usando el
            constructor por defecto
            Object persona =
            clase.getDeclaredConstructor().newInstance();

            // Obtener todos los métodos declarados en la clase
            Persona
            Method[] methods = clase.getDeclaredMethods();

            // Mostrar los nombres de los métodos
            System.out.println("Métodos de la clase Persona:");
            for (Method method : methods) {
                System.out.println("Método: " + method.getName());
            }

            // Obtener el método setNombre y ejecutarlo para
            establecer el nombre de la persona
            Method setNombre = clase.getDeclaredMethod("setNombre",
            String.class);
            setNombre.invoke(persona, "Juan");

            // Obtener el método getNombre y ejecutarlo para obtener
            el nombre de la persona
            Method getNombre = clase.getDeclaredMethod("getNombre");
            String nombre = (String) getNombre.invoke(persona);
            System.out.println("El nombre de la persona es: " +
            nombre);

            // Acceder y modificar un campo privado llamado "edad"
            Field edad = clase.getDeclaredField("edad");
            edad.setAccessible(true); // Hacemos el campo accesible
            edad.setInt(persona, 30); // Establecemos el valor del
            campo "edad"

            // Obtener y mostrar el valor del campo privado "edad"
            int edadPersona = edad.getInt(persona);
            System.out.println("La edad de la persona es: " +
            edadPersona);

        } catch (ClassNotFoundException | NoSuchMethodException |
            IllegalAccessException |
                InvocationTargetException | InstantiationException |
                NoSuchFieldException e) {
            e.printStackTrace();
        }
    }
}
```



PARA SABER MÁS

Aquí tienes la documentación sobre el API de Java para la reflexión:



VÍDEO DE INTERÉS

En este videotutorial se explica detalladamente el tema de la introspección en Java (pon atención entre los minutos 3 y 13):



EJEMPLO PRÁCTICO

Nuestro jefe nos solicita ayuda porque tienen una aplicación que gestiona información en tablas y la quieren migrar a Hibernate. Para ello, nos solicitan un componente para gestionar los datos de productos en una base de datos relacional que implemente un ORM y permita realizar operaciones CRUD sobre la base de datos de productos.

En primer lugar, gestionaremos las dependencias de Maven para importar.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.productoshibernate </groupId>
```

```
<artifactId>ProductosHibernate</artifactId>
<version>0.0.1-SNAPSHOT</version>

<dependencies>
    <dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>6.1.6.Final</version>
    </dependency>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.31</version>
    </dependency>
    <dependency>
        <groupId>javax.activation</groupId>
        <artifactId>activation</artifactId>
        <version>1.1.1</version>
    </dependency>
</dependencies>

</project>
```

A partir de este pom, definiremos la entidad producto para alcanzar la encapsulación del componente con propiedades ligadas, como hemos visto en el apartado 1:

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.Serializable;

@Entity
public class Producto implements Serializable {
    @Id
    @GeneratedValue
    private int id;
    private String nombre;
    private double precio;

    private transient final PropertyChangeSupport changeSupport = new
PropertyChangeSupport(this);

    public Producto() {}

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    // Propiedades simples
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
```

```
public String getNombre() { return nombre; }
public void setNombre(String nombre) {
    String oldNombre = this.nombre;
    this.nombre = nombre;
    changeSupport.firePropertyChange("nombre", oldNombre,
nombre);
}

public double getPrecio() { return precio; }
public void setPrecio(double precio) {
    double oldPrecio = this.precio;
    this.precio = precio;
    changeSupport.firePropertyChange("precio", oldPrecio,
precio);
}

// Gestión de eventos
public void addPropertyChangeListener(PropertyChangeListener
listener) {
    changeSupport.addPropertyChangeListener(listener);
}

public void removePropertyChangeListener(PropertyChangeListener
listener) {
    changeSupport.removePropertyChangeListener(listener);
}
}
```

Después, crearemos el fichero de configuración de Maven (hibernate.cfg.xml):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Database connection settings -->
        <property
name="connection.driver_class">com.mysql.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/productostienda</pr
operty>
        <property name="connection.username">root</property>
        <property name="connection.password"></property>
        <property name="show_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">create</property>

        <!-- Mapeo de las clases -->
        <mapping class="Entidades.Producto" />

    </session-factory>
</hibernate-configuration>
```


Por último, hacemos un main que pruebe nuestro componente:

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class Principal {
    public static void main(String[] args) {
        SessionFactory factory = new
Configuration().configure().addAnnotatedClass(Producto.class).buildSe
ssionFactory();

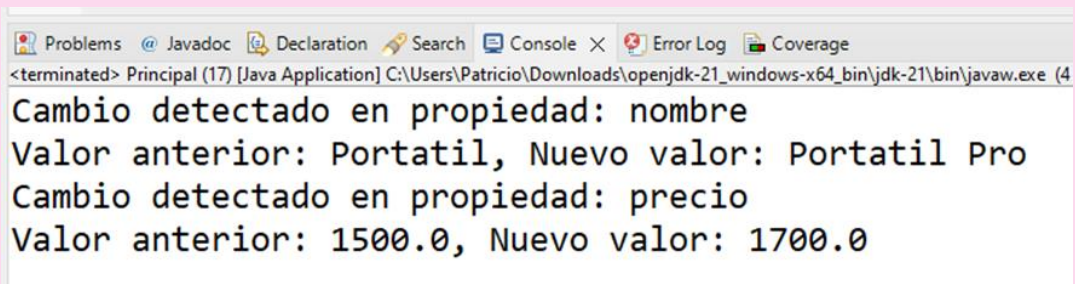
        try (Session session = factory.openSession()) {
            session.beginTransaction();

            // Crear un producto y registrar un listener
            Producto producto = new Producto("Portatil", 1500.00);
            producto.addPropertyChangeListener(evt -> {
                System.out.println("Cambio detectado en propiedad: "
+ evt.getPropertyName());
                System.out.println("Valor anterior: " +
evt.getOldValue() + ", Nuevo valor: " + evt.getNewValue());
            });

            // Guardar producto en la base de datos
            session.save(producto);

            // Cambiar propiedades
            producto.setNombre("Portatil Pro");
            producto.setPrecio(1700.00);

            session.getTransaction().commit();
        } finally {
            factory.close();
        }
    }
}
```



```
<terminated> Principal (17) [Java Application] C:\Users\Patricio\Downloads\openjdk-21_windows-x64_bin\jdk-21\bin\javaw.exe (4)
Cambio detectado en propiedad: nombre
Valor anterior: Portatil, Nuevo valor: Portatil Pro
Cambio detectado en propiedad: precio
Valor anterior: 1500.0, Nuevo valor: 1700.0
```

Resultado de la ejecución.



EJEMPLO PRÁCTICO

Nuestro jefe nos solicita ayuda porque quiere migrar una aplicación que tenían con documentos a una base de datos documental nativa con BaseX. Esta aplicación se encarga de gestión de personal, por lo que nos pide que vayamos generando un componente para los empleados con retrospección.

En primer lugar, añadiremos al pom las dependencias correspondientes:

```
<dependencies>
  <dependency>
    <groupId>org.basex</groupId>
    <artifactId>basex</artifactId>
    <version>9.7.3</version>
  </dependency>
</dependencies>
```

Luego, crearemos la clase que gestiona los empleados:

```
package empleadosxml;

public class Empleado {
    private String nombre;
    private String puesto;

    public Empleado() {}

    public Empleado(String nombre, String puesto) {
        this.nombre = nombre;
        this.puesto = puesto;
    }

    // Getters y Setters
    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getPuesto() {
        return puesto;
    }

    public void setPuesto(String puesto) {
        this.puesto = puesto;
    }

    @Override
    public String toString() {
        return "Empleado{" +
```

```
        "nombre='" + nombre + '\'' +  
        ", puesto='" + puesto + '\'' +  
        '}' ;  
    }  
}
```

Ahora, crearemos la clase que interactúa con la base de datos BaseX.

```
package empleadosxml;  
  
import org.basex.core.Context;  
import org.basex.core.BaseXException;  
import org.basex.core.cmd.Add;  
import org.basex.core.cmd.Close;  
import org.basex.core.cmd.CreateDB;  
import org.basex.core.cmd.XQuery;  
  
public class ManejadorBBDD {  
    private final Context context;  
  
    public ManejadorBBDD() {  
        this.context = new Context();  
    }  
  
    public void crearBaseDeDatos(String nombreBaseDatos) {  
        try {  
            new CreateDB(nombreBaseDatos).execute(context);  
            System.out.println("Base de datos '" + nombreBaseDatos +  
"" creada.");  
        } catch (BaseXException e) {  
            System.err.println("Error al crear la base de datos: " +  
e.getMessage());  
        }  
    }  
  
    public void agregarDocumento(String nombreDocumento, String  
contenidoXML) {  
        try {  
            new Add(nombreDocumento, contenidoXML).execute(context);  
            System.out.println("Documento agregado: " +  
nombreDocumento);  
        } catch (BaseXException e) {  
            System.err.println("Error al agregar documento: " +  
e.getMessage());  
        }  
    }  
  
    public String realizarConsulta(String consultaXQuery) {  
        try {  
            return new XQuery(consultaXQuery).execute(context);  
        } catch (BaseXException e) {  
            System.err.println("Error al realizar la consulta: " +  
e.getMessage());  
            return null;  
        }  
    }  
}
```

```
public void cerrar() {  
    try {  
        new Close().execute(context);  
    } catch (BaseXException e) {  
        System.err.println("Error al cerrar la base de datos: " +  
e.getMessage());  
    } finally {  
        context.close();  
    }  
}
```

Crearemos una Clase para hacer retrospección sobre el componente:

```
package empleadosxml;  
  
import java.lang.reflect.Field;  
import java.lang.reflect.Method;  
  
public class Retrospección {  
    public static void analizarClase(Class<?> clazz) {  
        System.out.println("Análisis de la clase: " +  
clazz.getName());  
  
        // Inspeccionar campos (propiedades)  
        System.out.println("Propiedades:");  
        for (Field field : clazz.getDeclaredFields()) {  
            System.out.println("- " + field.getName() + " (Tipo: " +  
field.getType().getSimpleName() + ")");  
        }  
  
        // Inspeccionar métodos  
        System.out.println("\nMétodos:");  
        for (Method method : clazz.getDeclaredMethods()) {  
            System.out.println("- " + method.getName());  
        }  
    }  
}
```

Por último, haremos la clase Principal que interactúe con la entidad e implemente la retrospección:

```
package empleadosxml;  
  
public class Principal {  
    public static void main(String[] args) {  
        ManejadorBBDD manejador = new ManejadorBBDD();  
  
        try {  
            // Crear la base de datos  
            manejador.crearBaseDeDatos("Empleados");  
  
            // Agregar documentos XML  
            manejador.agregarDocumento("emplead01.xml",  
"<empleado><nombre>Ana</nombre><puesto>Desarrollador</puesto></emplea  
do>");  
        }  
    }  
}
```

```

        manejador.agregarDocumento("empleado2.xml",
"<empleado><nombre>Juan</nombre><puesto>Analista</puesto></empleado>"
);

        manejador.agregarDocumento("empleado3.xml",
"<empleado><nombre>María</nombre><puesto>Gerente</puesto></empleado>"
);

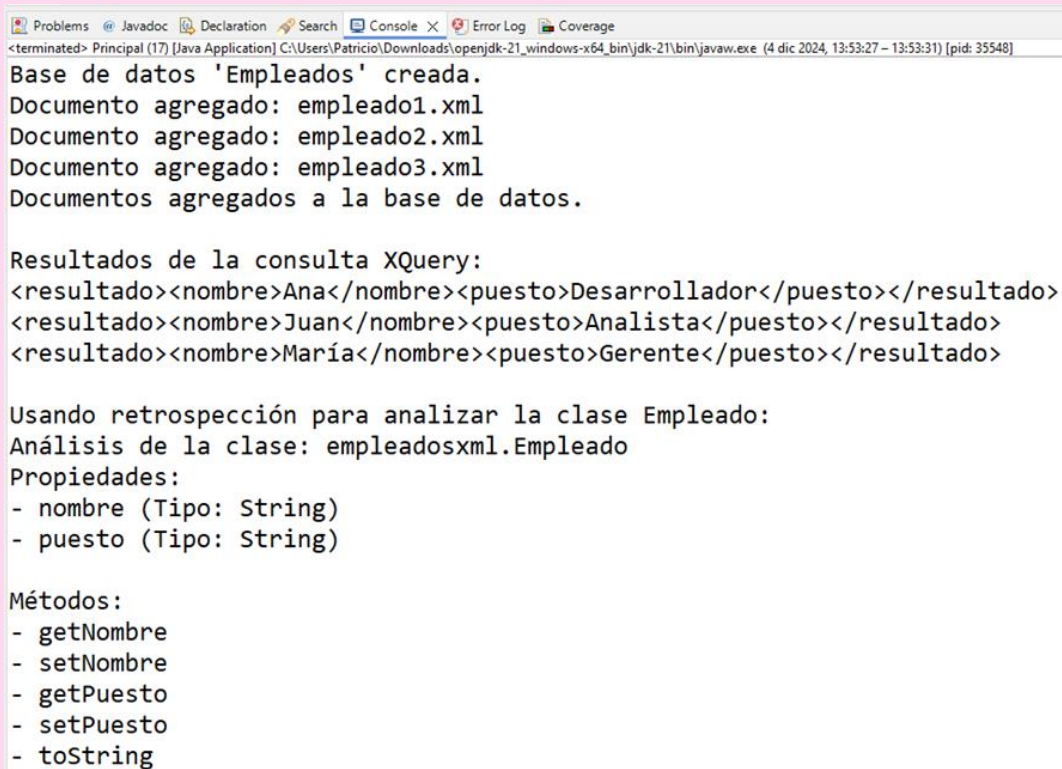
        System.out.println("Documentos agregados a la base de
datos.");

        // Realizar consulta XQuery
        String consulta = "for $e in //empleado return
<resultado><nombre>{$e/nombre/text()}</nombre><puesto>{$e/puesto/text
()}</puesto></resultado>";
        String resultado = manejador.realizarConsulta(consulta);

        // Mostrar los resultados
        System.out.println("Resultados de la consulta XQuery:");
        System.out.println(resultado);

        // Analizar dinámicamente la clase Empleado
        System.out.println("\nUsando retrospección para analizar
la clase Empleado:");
        Retrospección.analizarClase(Empleado.class);
    } finally {
        // Cerrar la conexión con BaseX
        manejador.cerrar();
    }
}
}

```



Problems Javadoc Declaration Search Console X Error Log Coverage

<terminated> Principal (17) [Java Application] C:\Users\Patricio\Downloads\openjdk-21_windows-x64_bin\jdk-21\bin\javaw.exe (4 dic 2024, 13:53:27 - 13:53:31) [pid: 35548]

Base de datos 'Empleados' creada.
Documento agregado: empleado1.xml
Documento agregado: empleado2.xml
Documento agregado: empleado3.xml
Documentos agregados a la base de datos.

Resultados de la consulta XQuery:
<resultado><nombre>Ana</nombre><puesto>Desarrollador</puesto></resultado>
<resultado><nombre>Juan</nombre><puesto>Analista</puesto></resultado>
<resultado><nombre>María</nombre><puesto>Gerente</puesto></resultado>

Usando retrospección para analizar la clase Empleado:
Análisis de la clase: empleadosxml.Empleado
Propiedades:
- nombre (Tipo: String)
- puesto (Tipo: String)

Métodos:
- getNombre
- setNombre
- getPuesto
- setPuesto
- toString

Resultado de la ejecución.

2. PLATAFORMAS Y MODELOS DE COMPONENTES

Pretendes seguir usando Java como motor de tus aplicaciones, ya que tienes mucha experiencia y, además, tienes muchos desarrollos ya realizados con esta tecnología.

Quieres entender qué opciones te proporciona Oracle con Java para desarrollar y distribuir componentes en el sector hostelero. Te planteas si incorporar el estándar JavaBeans o CORBA a tus desarrollos. Al final, optas por realizar un JavaBean, empaquetarlo en .jar y, por último, realizar sus pruebas correspondientes para garantizar su funcionamiento.

Los modelos de componentes son los estándares encargados de definir la forma de las interfaces de los componentes y determinar los mecanismos de composición y comunicación entre ellos. Ejemplos de modelos son: COM/DCOM, JavaBeans y CORBA.

- **CORBA.** Estándar abierto creado por el OMG (Object Management Group) que se encarga de definir estándares para facilitar la interoperabilidad y portabilidad. Este estándar permite que componentes software escritos en distintos lenguajes de programación y que ejecutan en distintas máquinas puedan trabajar juntos.
- **JavaBeans.** Modelo de componentes creado por la compañía Sun Microsystems para la construcción de aplicaciones con el lenguaje de programación Java. Hoy en día es uno de los modelos de desarrollo de software basado en componentes con más aceptación. Permite la comunicación entre componentes a través de eventos. A los componentes del modelo se les denomina Beans y son componentes reutilizables que se pueden manipular mediante herramientas de desarrollo de aplicaciones. Entre las ventajas de JavaBeans cabe destacar:
 - Portabilidad.
 - La manipulación de los componentes constituye un proceso simple que se puede realizar con o sin herramienta de desarrollo.
 - Compatibilidad con otros modelos de componentes.
 - Comunicación para acceder a datos remotos a través de RMI, JavaIDL o HTTP.
- **COM.** Modelo de componentes creado por Microsoft que permite, al igual que CORBA, la interoperabilidad entre componentes independientemente de los lenguajes de programación en los que se hayan escrito y de las plataformas sobre las que corran. Aunque esto no fue así inicialmente, hoy en día existen versiones para trabajar con distintos sistemas operativos.

Las **plataformas de componentes** son aquellas que proporcionan un entorno de desarrollo y ejecución para los componentes, ofreciendo una implementación de los conceptos y mecanismos del modelo en el que se basan. Cada plataforma de componentes está basada en un modelo concreto, aunque normalmente ofrecen pasarelas a otros modelos y plataformas.

Algunas de las plataformas de desarrollo de componentes son ActiveX/OLE, Enterprise Beans y Orbix, que se apoyan en los modelos de componentes COM, JavaBeans y CORBA, respectivamente. Algunos de ellos son:

- **Enterprise Beans.** Se trata de una arquitectura para la creación de componentes de aplicaciones distribuidas orientadas a objetos, escritas en el lenguaje de programación Java. Sus principales características son:
 - Compatibilidad con el protocolo CORBA.
 - Facilidad de creación de aplicaciones, ya que no hay que preocuparse de conceptos de bajo nivel.
 - Las aplicaciones escritas con esta arquitectura serán transaccionales, escalables y multiusuarios.
 - Permite su configuración editando los parámetros con el lenguaje XML.
- **Orbix.** Se trata de una implementación del estándar CORBA que destaca por su alto rendimiento. Además, es multiprotocolo y multiplataforma, por lo que rompe la brecha entre sistemas operativos y lenguajes.
- **ActiveX/OLE (Object Linking and Embedding).** Se trata de un estándar para la definición de componentes creado por Microsoft y usado en ambientes Windows por lenguajes de programación como VisualBasic, C++, Delphi, entre otros.



PARA SABER MÁS

Lee esta introducción al modelo de componentes CORBA:





ENLACE DE INTERÉS

Conoce más a fondo cómo funcionan estas plataformas y, en concreto, profundiza sobre Java EE:



ENLACE DE INTERÉS

También es interesante que compruebes este manual de la programación de aplicaciones con CORBA:



2.1 Herramientas para el desarrollo de componentes

Un entorno de desarrollo o IDE, por sus siglas en inglés, para el desarrollo de componentes, es una aplicación visual que se utiliza para el desarrollo de aplicaciones a partir de componentes. Normalmente cuentan con los siguientes componentes:

- Paletas donde aparecen los iconos de componentes disponibles.
- Lienzo o contenedor donde se colocan los componentes.
- Editores para realizar la configuración de los componentes.
- Visores para localizar componentes según determinados criterios de búsqueda.
- Herramientas de control varias.

Por ejemplo, para crear componentes en Java (JavaBeans), se pueden usar los IDE NetBeans y Eclipse. A continuación, se verá cómo hacerlo con NetBeans.



ENLACE DE INTERÉS

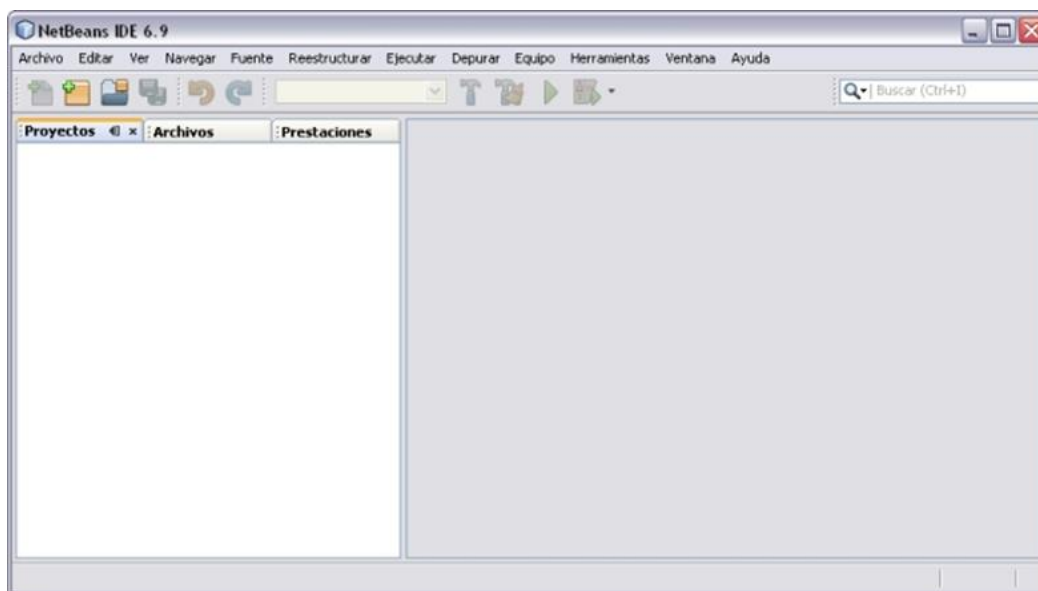
En esta web se puede obtener la información oficial sobre JavaBeans:



Este entorno de desarrollo está diseñado para construir componentes y aplicaciones portables entre distintas plataformas, ya que hace uso de la tecnología Java. El lenguaje de programación con el que se trabaja es Java.

NetBeans permite construir aplicaciones a partir de un conjunto de componentes de software, beneficiándose de las ventajas de la programación orientada a componentes que se mencionaban en el punto uno: reutilización de componentes, facilidad de extensión de las funcionalidades de las aplicaciones, etc.

Una vez instalado el entorno, aparecerá la página principal:



Entorno Netbeans.

Fuente: Elaboración propia

En la parte de la izquierda aparecerán tres pestañas: Proyectos, Archivos y Prestaciones.

Las aplicaciones en este entorno se denominan Proyectos, y estos, a su vez, contienen varios elementos (ficheros de código, etc.). Crear un nuevo proyecto es tan fácil como dirigirse al menú Archivo/Proyecto nuevo, seleccionar el tipo de proyecto a crear y la ubicación de este. Una vez creado, se puede observar cómo las fichas antes mencionadas se van rellenando de elementos:



Pestaña de proyectos.

Fuente: Elaboración propia

Lo siguiente que habría que hacer sería crear un fichero de código fuente, para lo que se debe pinchar sobre Source Files de la pestaña Proyectos y seguir el asistente, que consiste prácticamente en indicar el nombre del archivo, el proyecto al que pertenece, la extensión y la ubicación.

Una vez creado el archivo fuente, en la parte derecha de la ventana principal de NetBeans se encuentra el editor de código fuente.



ENLACE DE INTERÉS

Se puede descargar este IDE desde su web oficial, donde también se encuentra numerosa documentación:



Ahora vamos a ver cómo crear JavaBeans. Como se indicó anteriormente, un JavaBean es un objeto Java que presenta ciertas características:

- Posee un constructor sin argumentos.
- Las propiedades son accesibles con los métodos `get` y `set` estudiados anteriormente.
- Es serializable, es decir, implementa la interfaz `Serializable`.

El principal objetivo de los JavaBeans es ser contenedores de código que puedan ser empaquetados e incorporados a aplicaciones que necesiten de esa funcionalidad.

Del ejemplo anterior se deduce que los métodos `set` y `get` deben ser públicos para poder acceder a los atributos a través de dichos métodos. Sin embargo, las variables deben ser privadas para que no sean accedidas desde cualquier clase.

Para **crear componentes con NetBeans**, se deben seguir los siguientes pasos:

1. Realizar la clase Java que se va a convertir en componente, siguiendo los pasos descritos anteriormente.
2. Comprobar que está bien estructurada y sin errores.
3. Crear la clase `BeanInfo`, que contiene la información sobre las propiedades, eventos y métodos del componente. Para esto, se siguen los siguientes pasos:
 - Crear una clase que implemente la interfaz `BeanInfo` y nombrarla con el mismo nombre de la clase fuente seguido de `BeanInfo`. Ejemplo. `public class PersonaBeanInfo implements BeanInfo`. Esta interfaz proporciona los métodos necesarios para obtener información sobre las propiedades, eventos y métodos de un componente software.
 - Sobrescribir los métodos necesarios para devolver las propiedades, métodos y eventos.
4. Generar el archivo JAR.

En cada uno de los apartados de esta unidad se han ido viendo dichos pasos.



EJEMPLO PRÁCTICO

Queremos convertir todos nuestros modelos de objetos (siguiendo el estándar JPA) para poder utilizarlos como un JavaBean. Para ello, queremos comenzar con uno de los objetos que nuestros clientes manejan en el área de Recursos Humanos de sus cocinas, es decir, con el objeto Empleado.

¿Cómo crear ese JavaBean de un Empleado?

Será tan sencillo como usar la anotación `@Entity` y, además, hacer `Serializable` el objeto de empleado. El código quedaría tal y como sigue:

```
@Entity
public class Empleado implements Serializable{

    @Id
    private int id;
    private String nombre;
    private int salario;

    // Constructor sin argumentos
    public Empleado () {}

    // Constructor del JavaBean
    public Empleado (String nombre, int salario) {
        this.nombre= nombre;
        this.salario= salario;
    }

    //Getters y Setters
    public int getId() {
        return id;
    }
    public void setId( int id ) {
        this.id = id;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre( String nombre) {
        this.nombre= nombre;
    }
    public int getSalario() {
        return salario;
    }
    public void setSalario( int salario) {
        this.salario= salario;
    }
}
```



VÍDEO DE INTERÉS

Es recomendable visionar este videotutorial sobre JavaBeans y NetBeans, poniendo especial atención entre los minutos 2 y 12:



2.2 Empaquetado de componentes

El empaquetado de aplicaciones hace referencia al conjunto de ficheros de una aplicación que se agrupan en uno mayor (paquete) y que se obtiene cuando ha finalizado el desarrollo y se desea poner en marcha sobre un dispositivo. En este empaquetado se incluirían los componentes software desarrollados o reutilizados.

Estos paquetes están formados por:

- Los programas ejecutables de la aplicación.
- Las bibliotecas necesarias de las que depende, las cuales pueden haber sido enlazadas estática o dinámicamente.
- Los componentes a enlazar.
- Otros tipos de ficheros (como imágenes, bases de datos, ficheros de audio, traducciones y localizaciones, etc.).

Todo dentro del mismo fichero, formando un conjunto que el usuario percibe como un único archivo que contiene el programa, independientemente del número de ficheros que incluya.

Dentro de las distintas fases del ciclo de vida de una aplicación, el empaquetado se situaría de la siguiente forma:



Ciclo de vida del componente.

Fuente: Elaboración propia

Es decir, el proceso de empaquetado se lleva a cabo una vez que ha finalizado la fase de depuración y la aplicación está lista para su **distribución**.

Una de las **ventajas** de un correcto empaquetado software es que permite evitar problemas de dependencias a la hora de instalar y usar la aplicación, ya que cada paquete incorpora sus dependencias y la instalación o desinstalación de otro software no va a afectar a las dependencias de este paquete. En cuanto a las desventajas, cabe citar que estos paquetes ocupan más espacio en disco, sobre todo si incluyen bibliotecas.

Lo estudiado para las aplicaciones es **extensible a los componentes**. Una vez que estos han sido contruidos y se ha comprobado que su funcionalidad es la adecuada, deben ser empaquetados para su posterior utilización e incorporación en otras aplicaciones.

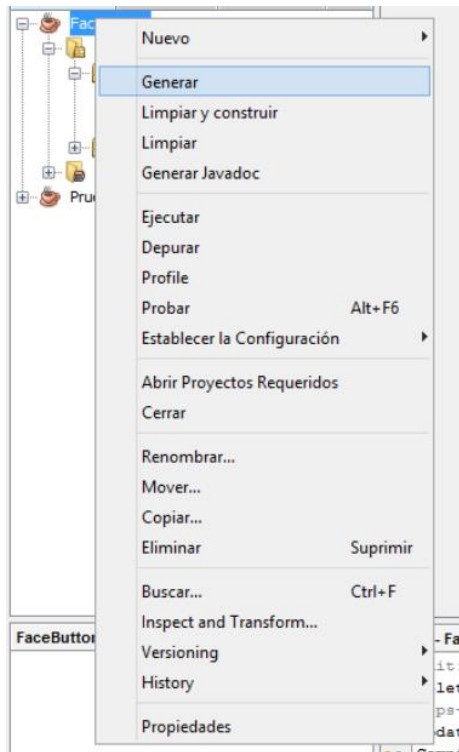
En el caso del lenguaje de programación Java, los paquetes tienen la **extensión .jar**.

.JAR es un tipo de archivo que permite ejecutar aplicaciones escritas en el lenguaje Java. Los archivos JAR están comprimidos con el formato ZIP y se les cambia su extensión a

.jar. Es el más popular entre los dispositivos móviles y en los sistemas operativos más importantes.

Para generar el **paquete jar desde NetBeans**, seguimos los siguientes pasos:

1. Clic derecho encima del nombre del proyecto.
2. Seleccionar la opción Generar.



Empaquetado de Componente.

Fuente: Elaboración propia

3. Si todo ha ido correctamente, debe salir un mensaje en consola indicando que la generación del fichero empaquetado ha sido correcta:

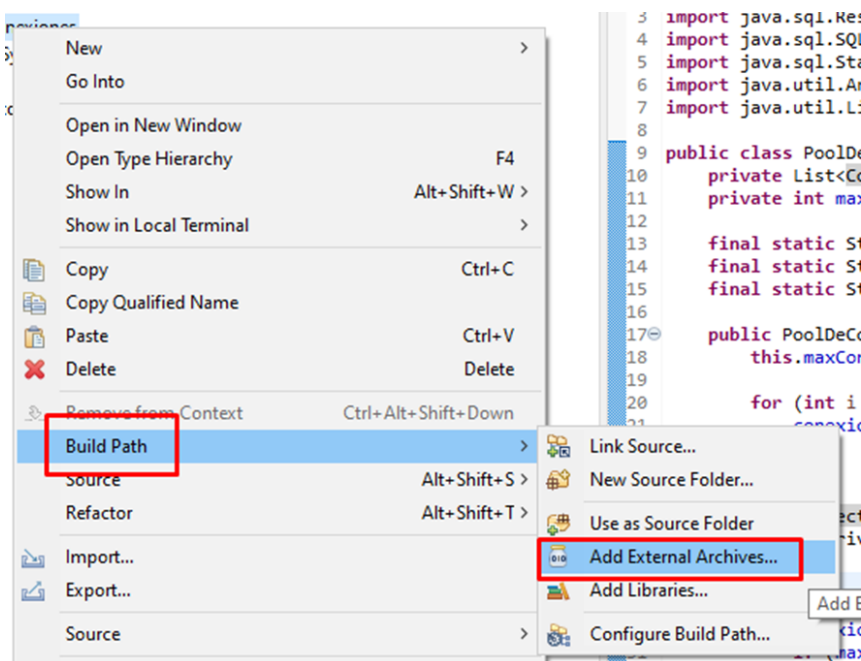
```
Salida - FaceButton (jar)
ant -f "C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton" jar
init:
Deleting: C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\build\build-jar.properties
deps-jar:
Updating property file: C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\build\build-jar.properties
Compiling 2 source files to C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\build\classes
Copying 4 files to C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\build\classes
compile:
Created dir: C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\dist
Copying 1 file to C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\build
Nothing to copy.
Building jar: C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\dist\FaceButton.jar
To run this application from the command line without Ant, try:
java -jar "C:\Users\HANNIBAL T\Documents\NetBeansProjects\FaceButton\dist\FaceButton.jar"
jar:
GENERACIÓN CORRECTA (total time: 0 seconds)
```

Resultado empaquetado componente.

Fuente: Elaboración propia.

2.3 Utilización de componentes

Una vez que un componente ha sido desarrollado y empaquetado, su integración en proyectos es sencilla. Además, ya hemos realizado este proceso en unidades anteriores, como cuando importábamos el JDBC. Para utilizar un componente que ha sido empaquetado como un archivo JAR, puedes agregar el JAR al classpath. Esto le indica al entorno dónde encontrar las clases y métodos del componente.



Agregando un JAR a nuestro proyecto.

Fuente: Elaboración propia.

2.4 Prueba y documentación de los componentes desarrollados

Ya se ha estudiado en ocasiones la importancia de la prueba y documentación en el desarrollo de aplicaciones. En el caso de los componentes desarrollados, la situación es similar, ya que, en cualquier caso, durante todas las fases del ciclo de vida del software, los desarrolladores pueden cometer innumerables errores humanos. Las pruebas permiten garantizar la calidad del software y representan una revisión final de las especificaciones, el diseño y la codificación. La prueba es, por tanto, el proceso de ejecución de un programa en busca de errores.

Una definición más técnica de las pruebas sería la siguiente: una actividad en la cual un sistema o uno de sus componentes se ejecuta en circunstancias previamente especificadas, siendo observados y analizados los resultados para evaluar determinados aspectos y comprobar si cumple o no los requisitos especificados. Se puede decir que una prueba ha tenido éxito si descubre un error no detectado hasta ese momento.

Una de las limitaciones en la realización de pruebas es que éstas no pueden asegurar la ausencia de errores, simplemente demuestran la existencia de estos.

A continuación, se muestra a modo de resumen las pruebas que se pueden realizar sobre el software. No se ahondará en ellas, puesto que ya se han estudiado a lo largo del ciclo en distintas unidades.

1. **Prueba de unidad:** se trata de las pruebas formales que permiten declarar que un módulo está listo y terminado.
2. **Prueba de integración:** los módulos individualmente probados se integran para comprobar sus interfaces (comunicación con otros módulos) en el trabajo conjunto.
3. **Prueba de validación:** se lleva a cabo cuando se han terminado las pruebas de integración. El software terminado se prueba como un conjunto para comprobar si cumple o no tanto los requisitos funcionales como los requisitos de rendimientos, seguridad, etc.
4. **Prueba de sistema:** una vez que el software cumple con los requisitos, se debe integrar con el resto del sistema. Esto implica, por ejemplo, instalarlo en el mismo sistema operativo y con las características de hardware que utilizará el cliente, para luego probar su funcionamiento en conjunto.
5. **Prueba de aceptación:** se instala el software en el propio entorno de explotación (entorno de producción), y el cliente comprueba que el software cumple con los requisitos establecidos.
6. **Pruebas de regresión:** su objetivo es determinar si los cambios que se han producido afectan a otras partes del software.
7. **Pruebas de volumen:** se utilizan para determinar los límites de datos que producen que el sistema falle. Además, se usan para determinar la carga máxima o volumen que el sistema es capaz de manejar durante un periodo de tiempo dado.
8. **Pruebas de estrés:** permiten encontrar errores debidos a recursos limitados, como poca memoria o espacio en disco. Su objetivo es analizar el comportamiento del sistema bajo condiciones que sobrecargan sus recursos.

9. **Pruebas de recuperación:** permiten asegurar que una aplicación o sistema se recupere de situaciones anómalas de hardware, software o red, con pérdidas de datos o fallos de integridad. Su principal objetivo es determinar la habilidad del sistema para recuperarse de un fallo.
10. **Pruebas de seguridad:** tienen como principal objetivo evaluar el correcto funcionamiento de los controles de seguridad del sistema para asegurar la integridad y confidencialidad de los datos.

En el primer apartado de esta unidad se vio que una de las desventajas de la programación orientada a componentes es el hecho de que el desarrollador no sabe quién utilizará finalmente el componente, ni cómo ni cuándo. Y esto afecta también al desarrollo de las pruebas, ya que en ocasiones el ambiente de operación que éste ha pensado puede ser diferente, y en ese caso es posible que haya obviado ciertas situaciones en los casos de prueba.

Es por esto por lo que es necesario hacer hincapié en que, normalmente, las pruebas de unidad son realizadas por el desarrollador, y las pruebas de integración, por el usuario final que integrará el componente en su aplicación. Sin embargo, hay que tener en cuenta que la mayoría de usuarios finales no cuentan con la formación y experiencia para realizar casos de prueba. A esto hay que añadir que, normalmente, cuando se suministra un componente, no se tiene acceso al código de éste, ya que aparece empaquetado en un archivo jar.

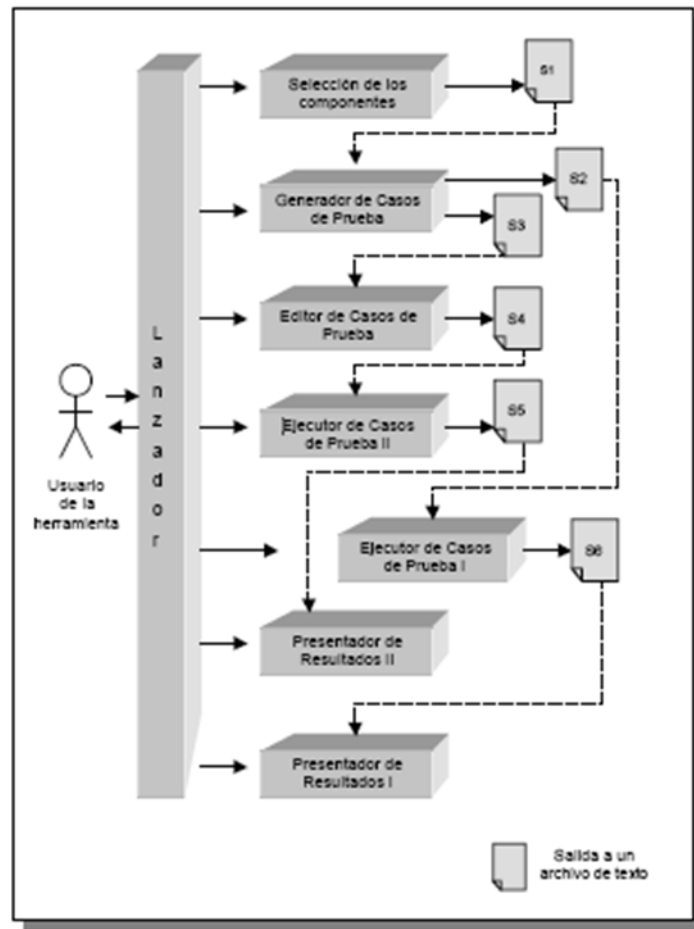


PARA SABER MÁS

Conoce más información sobre las estrategias de prueba de software en el siguiente enlace:



A pesar de estas desventajas, tanto el desarrollador como el destinatario del componente deben realizar las siguientes fases a la hora de planificar el proceso de pruebas:



Fases de pruebas.

Fuente: Elaboración propia

En cuanto a la documentación de las aplicaciones o componentes software, es un aspecto muy importante no solo en el desarrollo de la aplicación, sino también en el mantenimiento de la misma. La mayoría de los desarrolladores de software no dedican mucho tiempo a la documentación de sus aplicaciones porque no lo consideran importante, sin embargo, son muchas las ventajas de su uso, ya que, por ejemplo, la documentación no solo sirve a los usuarios finales de la aplicación, sino que también facilita la reutilización o modificación de código por parte de futuros desarrolladores. Y este punto es el más importante de cara a la programación orientada a componentes, ya que el objetivo del desarrollo de componentes es crear software empaquetado que pueda incorporarse en aplicaciones más grandes.

Al igual que se comentó para la documentación de las aplicaciones, en el caso de los componentes, debe realizarse simultáneamente a la construcción de éstos y terminar justo antes de la entrega del componente al usuario final.

Los tipos de documentación son los mismos que los considerados para las aplicaciones:

- **Interna:** documentación que se crea en el mismo código, ya puede ser en forma de comentarios o de archivos de información dentro del componente.

- **Externa:** documentación que se escribe en manuales o libros, totalmente ajena al componente en sí.

Una vez concluido el componente, los documentos que se deben entregar son una guía técnica, una guía de uso y una de integración.

- **La guía técnica:** donde se reflejan el diseño del componente, la codificación, siempre que se incluya y no se entregue empaquetado, y las pruebas realizadas para su correcto funcionamiento.
- **La guía de usuario o manual de referencia:** contiene la información necesaria para que los usuarios utilicen correctamente el componente.
- **La guía de integración:** es la guía que contiene la información necesaria para integrar el componente en una aplicación.



PARA SABER MÁS

Lee el último apartado de este documento, ya que incluye normas sobre cómo realizar una documentación de calidad:



ENLACE DE INTERÉS

Aquí comprenderás la importancia de la documentación en el desarrollo de una aplicación:



RESUMEN FINAL

En esta unidad se ha estudiado el concepto de componente y cómo aplicarlo a la programación, más concretamente en Java, a través de diversos mecanismos. Para ello, hemos visto los conceptos básicos, siendo uno de los conceptos necesarios el de persistencia de componentes y cómo implementarlo en nuestros desarrollos.

Se han presentado los diferentes métodos de creación de componentes en Java y se ha detallado para el caso de los JavaBeans, viendo cómo crearlos y generarlos posteriormente desde un entorno de desarrollo como NetBeans.

Para finalizar, se han recordado los conceptos de pruebas y documentación del software, particularizándolos en esta ocasión en el desarrollo de componentes.